

Nicolas Marques – CID2A

Vennes

32 périodes

Table des matières

Analyse 3
Réalisation 6
Program.cs 6
PlayGame.cs 7
StartProgram 7
Menu.cs 8
DisplayMenu 8
UserInputHandler 9
EnterVehicle 10
SearchVehicle 11
CallFindVehicle 12
Parking.cs 13
Display 13
CheckParkingSpot 14
AddVehicle 15
FindVehicule 16
FindVehicleReturn 17
StatParking 18
Voiture.cs 19
LicensingPlate 19
Constructeur 19
Ticket.cs 20
Constructeur 20
Calc 20
GetCurrentParkedTime 20
Tests 21
CheckParkingSpot_IsFree_Result_L 21
GetTotalPaidPrice_Is0_Result_0 21
StatParking_IsDaily_Result_Daily 21
Conclusion 22
Bilan personnel 22

Utilisiation de l’IA 22
Bilan des fonctionnalités 22

Analyse

Le projet est fait sur C# console. C'est un "simulateur" de parking. Un projet centré sur la P.O.O.

1 : Un système "d'horaire" est présent pour avoir l'heure d'entrée, de sortie du véhicule puis calculé son tarif basé sur sa durée de stationnement.

2 : L'utilisateur peut ajouter des véhicules en entrant une plaque d'immatriculation sous ce format "***2 lettres*** – ***jusqu'à 6 chiffres***". Puis un ticket va automatiquement être attribuer au véhicule (***son heure d'entrée, la place de parking, puis le tarif horaire***).

3 : Un système de recherche fonctionnant comme ce ci : L'utilisateur entre soit une plaque d'immatriculation ou un numéro de place.

4 : L'utilisateur peut chercher un véhicule en utilisant le système de recherche. Si une correspondance est trouvée, ***afficher sa plaque d'immatriculation, heure d'entré, durée de stationnement, prix actuel, et son numéro de place***. Si la plaque ou le numéro de place n'a aucun résultat, afficher un message pour dire à l'utilisateur que aucun résultat n'a pu être trouvé.

5 : L'utilisateur aura la possibilité de sortir un véhicule du parking en utilisant le système de recherche. Si une correspondance est trouvée, d'abord demandé à l'utilisateur s'il est sûr de vouloir libérer la place. Si oui, ***afficher son heure d'entrée et de sortie et le montant payé***. Si non, annulé l'action. En revanche si aucune correspondance est trouvée, afficher un message pour dire à l'utilisateur que aucun résultat n'a pu être trouvé.

6 : Afficher l'état du parking de manière visuel. Afficher les places occupées et libres. Pour que l'utilisateur ce repère plus facilement.

7 : Avoir des statistiques du jour. Par exemple le pourcentage du parking occupées, le montant total payé, la durée de stationnement de tous les véhicules actuels, la durée moyenne de stationnement, etc...

8 : Avoir un historique des transactions. Afficher l'heure d'entrée d'un véhicule avec sa plaque d'immatriculation et son numéro de place écrit à coter. Puis afficher son heure de sortie avec les mêmes infos que l'heure d'entrée mais avec le montant payé en plus.

1 : **Créer une classe statique pour les méthodes du projet.** Utilisé un timer qui s'incrémentera chaque seconde. Une seconde sera égale à une minute.

2 : Crée 2 classes non-statiques. 1 pour une classe ticket (***contenant les infos du tickets***).1 pour le véhicule (sa *plaque d'immatriculation*, et son ticket.). Utiliser la composition/association d'objet pour avoir le ticket comme attributs de la classe voiture.
Puis ensuite Utiliser la classe statique pour crée une méthode qui a pour paramètre un string (pour la plaque d'immatriculation). On va vérifier si le numéro de plaque entrée par l'utilisateur est cohérent.
Si oui, on va créer un nouveau véhicule avec la plaque entrée par l'utilisateur, puis un ticket avec ***ses infos***.
Sinon, on va afficher un message pour dire à l'utilisateur que ce n'est pas la bonne façon pour créer un véhicule. Puis montrer à l'utilisateur un exemple de comment doit être écrit la plaque d'immatriculation.

3 : Utiliser la classe statique pour créer une méthode permettant la recherche de véhicule. Cette méthode aura comme paramètre une variable de type string, puis une variable de type booléenne (pour dire ce que l'utilisateur veut faire. Chercher un véhicule, ou le faire sortir). On va vérifier si le string est cohérent pour savoir si c'est une plaque d'immatriculation, si cela ne fonctionne pas, essayer de le transformer en type INT (pour savoir si c'est un numéro de place), si ça ne fonctionne pas, appeler une autre méthode (soit pour afficher les infos du véhicule, soit le faire sortir. Dont l'utilité du paramètre booléen) puis lui donner comme paramètre un texte écrit comme ce ci "nothing found".
Cette méthode appellera une autre méthode en lui donnant comme paramètre la plaque (si l'utilisateur avait entrée un numéro de place, on aurait cherché dans le parking la place en question puis récupéré son numéro de plaque qui l'occupe) si une correspondance a été trouvé.

4 : Utiliser la classe statique pour créer une nouvelle méthode pour afficher les informations du véhicule. La méthode aura comme paramètre une variable de type string. On vérifie si le string est un numéro de plaque cohérent.

Si oui, *récupérer les informations* puis les afficher.

Si non, vérifier si le string est égal à "nothing found". Si c'est le cas, afficher un message pour dire à l'utilisateur que aucun résultat n'a pu être trouvé. Si ce n'est pas le cas, afficher un message d'erreur comme "Un problème c'est produit." ;

5 : Utiliser la classe statique pour créer une nouvelle méthode pour faire sortir un véhicule du parking. La méthode aura comme paramètre une variable de type string. On vérifie si le string est un numéro de plaque cohérent.

Si oui, demander à l'utilisateur s'il est sûr de vouloir faire quitter ce véhicule. Si l'utilisateur répond oui, ***afficher ses informations***. Sinon, annulé l'action.

Si non, vérifier si le string est égal à "nothing found". Si c'est le cas, afficher un message pour dire à l'utilisateur que aucun résultat n'a pu être trouvé. Si ce n'est pas le cas, afficher un message d'erreur comme "Un problème c'est produit." ;

6 : Utiliser la classe statique pour créer une nouvelle méthode pour afficher le parking. Le parking sera un tableau multi-dimensionnel, avec une taille fixe.

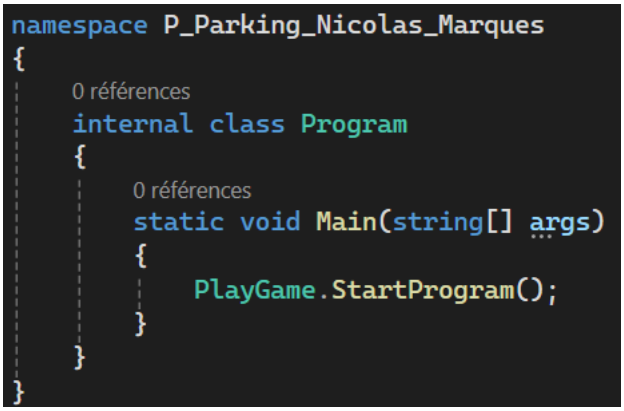
7 : Utiliser la classe statique pour créer une nouvelle méthode pour afficher diverse stats et information.

8 : Utiliser la classe statique pour créer une nouvelle méthode pour afficher l'historique des transactions.

Réalisation

Program.cs

Dans le fichier « program » il est principalement vide. La seule action qu'elle effectue c'est d'appeler une classe nommée « PlayGame ».



PlayGame.cs

Dans cette classe il se situe 3 méthodes. Elle exécute une loop qui permet au programme de tourner continuellement jusqu'à ce que l'utilisateur décide d'arrêter le programme.

StartProgram

```
private static bool continueProgram = Menu.ContinueProgram;

/// <summary>
/// Va commencer le programme, afficher le menu etc...
/// </summary>
1 référence
public static void StartProgram()
{
    InitiateParking();

    while (continueProgram)
    {
        Menu.DisplayMenu();
        continueProgram = Menu.ContinueProgram;
    }
}
```

Dans cette méthode on commence par appelé « InitiateParking » (qui permet simplement d'initialiser le parking) en premier. Ensuite on commence une boucle avec « continueProgram » (un booléen déclaré tout en haut de la capture) qui va faire en sorte que le menu et continuellement afficher puis de mettre à jour la variable « continueProgram ».

Menu.cs

Cette classe statique permet d'afficher le menu puis gérer les inputs de l'utilisateur. La deuxième plus grande classe de ce projet.

DisplayMenu

```
/// <summary>
/// Afficher le menu
/// </summary>
4 références
public static void DisplayMenu()
{
    int optionIndex = 0;
    Console.WriteLine("=== Menu principal ===");
    foreach (var item in menuText)
    {
        optionIndex++;
        Console.WriteLine($"{optionIndex}: {item}");
    }
    Console.Write(enterText);
    UserInputHandler();
}
```

Une simple méthode permettant d'afficher le menu. Une boucle foreach pour afficher chaque option (stocker dans un tableau private au début du code) disponible pour l'utilisateur. « optionIndex » permet à l'utilisateur de savoir quelle numéro entrer pour utiliser cette option-ci ou celle-là.

UserInputHandler

```
/// <summary>
/// Vérifie ce que l'utilisateur choisie
/// </summary>
2 références
private static void UserInputHandler()
{
    string userString = Console.ReadLine();
    int chosenNumber = 0;
    while (!int.TryParse(userString, out chosenNumber))
    {
        CheckCorrectInput(userString);
        userString = Console.ReadLine();
    }
    CheckInRange(chosenNumber);
}
```

Elle est souvent appelée dans le code. Elle va gérer ce que l'utilisateur entre comme option (ex : « 5 » pour l'option numéro 5). Si la saisie de l'utilisateur ne peut pas être converti en int, on va supprimer son input (grâce à la méthode « CheckCorrectInput » qui est utiliser seulement pour effacer la saisie du joueur pour avoir une console propre). Car effacer l'entièreté de la console serait problématique vu qu'on aurait besoin de re-appeler « DisplayMenu » simplement pour afficher un texte « Entrer votre choix : ».

Si c'est bien un nombre, on va vérifier s'il est trop grand ou petit. Si oui on va dire à l'utilisateur de choisir un nombre dans la plage de valeur correcte. On va attendre 1.5 secondes (pour laisser l'utilisateur le temp de lire le message) avant d'appeler « UserInputHandler » une fois de plus.

Si tout est bon, l'option choisi par l'utilisateur va être exécuté grâce a une méthode qui contient un simple « switch case » pour appeler la méthode correspondante à l'option.

EnterVehicle

```
/// <summary>
/// Permet à l'utilisateur d'entrer un véhicule
/// </summary>
1 référence
private static void EnterVehicle()
{
    ClearConsoleAndPosition();
    Console.Write("Ajouter un véhicule\nEntrer une plaque sous ce format (2LettresMajuscules-1à6Chiffres)\nEntrer une valeur : ");
    string userLicensePlate = Console.ReadLine();
    if (userLicensePlate != null)
    {
        new Voiture(userLicensePlate);
    }
    ReturnMultipleLines();
    DisplayMenu();
}
```

Ceci est une des options disponibles. On va nettoyer la console (pour avoir simplement le texte expliquant quel format est accepté pour que ça soit plus lisible) puis demander d'entrer une plaque. Si la saisie n'est pas vide, on va créer une voiture (la logique pour vérifier si la plaque est bonne ce déroule dans la classe « voiture ». Qui va informer au l'utilisateur si la voiture a bien été ajouté ou non). Faire des retours à la lignes avec « ReturnMultipleLines » (pour plus de lisibilité) puis réafficher le menu.

SearchVehicle

```
/// <summary>
/// Va rechercher un véhicule via son numéro de place de parking ou sa plaque d'immatriculation
/// </summary>
/// <param name="removeCar">Si on doit retirer le véhicule quand il est trouver</param>
2 références
private static void SearchVehicle(bool removeCar)
{
    ClearConsoleAndPosition();
    if (removeCar)
    {
        Console.Write("Retirer un véhicule\nEntrer une plaque sous ce format (2LettresMajuscules-1à6Chiffres)\nou son numéro de place\nEntrer une valeur : ");
    }
    else
    {
        Console.Write("Rechercher un véhicule\nEntrer une plaque sous ce format (2LettresMajuscules-1à6Chiffres)\nou son numéro de place\nEntrer une valeur : ");
    }

    int spotNumber = 0;
    string userLicensePlate = Console.ReadLine();
    if (userLicensePlate != null)
    {
        if(int.TryParse(userLicensePlate, out spotNumber))
        {
            CallFindVehicle(userLicensePlate, spotNumber, false, removeCar);
        }
        else
        {
            if (Regex.IsMatch(userLicensePlate, @"^[A-Z]{2}-\d{1,6}$"))
            {
                CallFindVehicle(userLicensePlate, spotNumber, true, removeCar);
            }
        }
    }
    DisplayMenu();
}
```

Une des autres options disponibles. Comme son nom l'indique, elle va rechercher un véhicule. Ducoup cette méthode essaye d'abord de transformer la saisie de l'utilisateur en nombre, si ça fonctionne, ça veut dire que c'est un numéro de place. Sinon, on essaye de faire un match (qui utilise le format des plaques d'immatriculation), si ça fonctionne, c'est une plaque, sinon, l'utilisateur a entré une valeur invalide.

CallFindVehicle

```
/// <summary>
/// Va appeller la méthode "CallFindVehicle" de la classe "Parking"
/// </summary>
/// <param name="userLicensePlate">La possible plaque entrer par l'utilisateur</param>
/// <param name="spotNumber">La possible place de parking enter par l'utilisateur</param>
/// <param name="checkLicensePlate">Si on doit vérifier la plaque ou le numéro de place</param>
/// <param name="removeCar">Si on doit retirer le véhicule quand trouver</param>
2 références
private static void CallFindVehicle(string userLicensePlate, int spotNumber, bool checkLicensePlate, bool removeCar)
{
    bool keepVehicle = true;
    if (removeCar)
    {
        keepVehicle = UserIsSure(true);
        if (keepVehicle)
        {
            Console.WriteLine("Action annulé");
        }
        else
        {
            Parking.FindVehicule(userLicensePlate, spotNumber, checkLicensePlate, removeCar);
        }
    }
    else
    {
        Parking.FindVehicule(userLicensePlate, spotNumber, checkLicensePlate, removeCar);
        ReturnMultipleLines();
    }

}
```

Cette méthode va d'abord vérifier si on veut retirer un véhicule. Si c'est le cas on va demander à l'utilisateur s'il est sûr de ça décision (en appelant « UserIsSure). S'il choisit non, on lui informe que l'action a été annulé. S'il choisit oui, alors on va chercher une correspondance pour supprimer le véhicule entrer (la méthode « FindVehicule »).

Cependant, si l'utilisateur voulait simplement afficher des informations d'un véhicule, on va donc rien faire de spécial puis appelé « FindVehicule ».

Parking.cs

Display

```
/// <summary>
/// Affiche le parking
/// </summary>
3 références
public static void Display()
{
    freeSpot = 0;
    usedSpot = 0;
    for (int i = 0; i < parking.Length; i++)
    {
        int currentSpotNumber = i + 1;
        if(currentSpotNumber % 5 == 0 && currentSpotNumber > 0)
        {
            ParkingSpot(true, currentSpotNumber, parking[i]);
        }
        else
        {
            ParkingSpot(false, currentSpotNumber, parking[i]);
        }
    }
    StatParking(false, false);
}
```

Cette méthode permet d'afficher le parking avec ses statistiques. Elle appelle cependant « ParkingSpot » qui va être celui qui va écrire chaque place de parking individuellement avec un certain style (ex : « |02:L||03:X| »).

CheckParkingSpot

```
/// <summary>
/// Permet d'avoir un symbole pour l'état de la place de parking.
/// </summary>
/// <param name="spotStatus">Pour savoir l'état du parking (libre ou occupé)</param>
/// <returns>Retourne le symbole de l'état du parking.
/// L = Libre
/// X = occupé
/// E = erreur/inconnue
/// </returns>
4 références
private static char CheckParkingSpot(int spotStatus)
{
    char spotSymbole;
    switch (spotStatus){
        case 0:
            spotSymbole = 'L';
            freeSpot++;
            usedSpot = totalSpot - freeSpot;
            break;
        case 1:
            spotSymbole = 'X';
            usedSpot = totalSpot - freeSpot;
            break;
        default:
            spotSymbole = 'E';
            usedSpot = totalSpot - freeSpot;
            break;
    }
    return spotSymbole;
}
```

Cette méthode (appeler par « ParkingSpot ») va s'occuper de gérer l'affichage du symbole en fonction de la valeur de la place de parking. Augmenté le nombre de place libre, le nombre de place occupé, etc...

AddVehicle

```
/// <summary>
/// Va implémenter le véhicule dans le parking
/// </summary>
/// <param name="vehicle">Le véhicule qu'on doit entrer dans le parking</param>
1 référence
public static void AddVehicle(Voiture vehicle)
{
    int randomSpot = RandomNumber.GenerateNumber(minParkingIndex, maxParkingIndex);
    if(freeSpot > 0)
    {
        while(parking[randomSpot] != 0)
        {
            randomSpot = RandomNumber.GenerateNumber(minParkingIndex, maxParkingIndex);
        }
        parking[randomSpot] = 1;
        randomSpot += 1;
        Ticket ticket = new Ticket(randomSpot, 1);
        //Thread.Sleep(100);
        vehicle.ReceiveTicket(ticket);
        parkedCarList.Add(vehicle);
    }
}
```

C’est ici qu’on rajoute une voiture dans le parking. La place de parking est choisie de manière aléatoire. On vérifie si la place est libre, si oui, on change la valeur de cette place pour qu’elle soit occupé. On créer et attribut un ticket pour la voiture. Puis on ajoute la voiture dans la liste des voitures qui sont parké.

FindVehicule

```
/// <summary>
/// Va rechercher un véhicule via l'input de l'utilisateur
/// </summary>
/// <param name="licensePlate">La plaque d'immatriculation possiblement entrer par l'utilisateur</param>
/// <param name="spotNumber">Le numéro de la place de parking possiblement entrer par l'utilisateur</param>
/// <param name="checkLicensePlate">Si on veut checker la plaque d'immatriculation ou son numéro de place de parking</param>
/// <param name="removeCar">Si on veut retirer la voiture ou simplement y afficher ses statisiques</param>
2 références
public static void FindVehicule(string licensePlate, int spotNumber, bool checkLicensePlate, bool removeCar)
{
    foreach(Voiture car in parkedCarList)
    {
        bool foundCorrectCar = FindVehicleCondition(car, licensePlate, spotNumber, checkLicensePlate, removeCar);
        if (foundCorrectCar)
        {
            return;
        }
        else
        {
            foundCorrectCar = FindVehicleCondition(car, licensePlate, spotNumber, checkLicensePlate, removeCar);
        }
    }
    Console.WriteLine("Aucune correspondance n'a pu être trouver.");
}
```

Ici on va rechercher un véhicule via sa plaque d'immatriculation ou son numéro de place. La méthode « FindVehicleCondition » va simplement vérifier s'il trouve une correspondance ou non. Si aucune correspondance est trouvé, on l'informe à l'utilisateur.

FindVehicleReturn

```
/// <summary>
/// Va retourner un résultat dépendant si on doit afficher ou retirer le véhicule
/// </summary>
/// <param name="car">La voiture</param>
/// <param name="removeCar">Si on veut retirer la voiture ou simplement y afficher ses statisiques</param>
2 références
private static void FindVehicleReturn(Voiture car, bool removeCar)
{
    if (removeCar)
    {
        parkedCarList.Remove(car);
        carList.Add(car);
        car.ticketCar.Calc();
        parking[car.ticketCar.SpotNumber-1] = 0;
        Console.WriteLine(car);
    }
    else
    {
        Console.WriteLine(car);
    }
}
```

Appeler par la méthode « FindVehicleCondition » (si une correspondance a été trouvé) elle permet d'effectuer une action. Soit afficher les informations du véhicule, soit le faire sortir du parking.

StatParking

```
/// <summary>
/// Affiche diverse statistiques du parking
/// </summary>
/// <param name="isDailyStat">Permet de savoir si on veut les statistiques du jour ou simplement les statistiques du parking</param>
3 références
public static void StatParking(bool isDailyStat, bool isTransactionHistory)
{
    if (isDailyStat)
    {
        WriteStats(isDailyStat, isTransactionHistory);
    }
    else
    {
        WriteStats(isDailyStat, isTransactionHistory);
    }
}
```

Cette méthode va soit afficher les statistiques du parking (action automatique lors de l'affichage du parking). Les statistiques quotidiennes (action de l'utilisateur) ou l'historique des transactions (action de l'utilisateur).

Voiture.cs

LicensingPlate

```
public string LicensingPlate
{
    get
    {
        return _licensingPlate;
    }
    private set
    {
        if(Regex.IsMatch(value, @"^[A-Z]{2}-\d{1,6}$"))
        {
            Debug.WriteLine("Valid");
            _licensingPlate = value;
            IsValid = true;
            Console.WriteLine($"Votre voiture ({LicensingPlate}) à été ajouté au parking!");
        }
        else
        {
            Debug.WriteLine("Invalid");
            IsValid = false;
            Console.WriteLine($"Le format est incorrecte. La voiture n'a pas été ajouté.");
        }
    }
}
```

C'est ici qu'on vérifie si la plaque d'immatriculation est écrite dans un format correct ou non. Puis informer l'utilisateur si son véhicule a été ajouté au parking ou non.

Constructeur

```
public Voiture(string licensingPlate)
{
    LicensingPlate = licensingPlate;
    if (IsValid)
    {
        Parking.AddVehicle(this);
    }

}
```

Le constructeur. Si le véhicule est valid, on l'ajoute automatiquement au parking, sinon, on ne fait rien.

Ticket.cs

Constructeur

```
public Ticket(int spotNumber, int hourlyRate)
{
    EnterTime = DateTime.Now;
    isParked = true;

    SpotNumber = spotNumber;
    HourlyRate = hourlyRate;
}
```

On va mettre l'heure d'entrée à l'heure actuel. Lui donné son numéro de place, son tarif horaire, puis lui mettre qu'il est parké (son véhicule parent).

Calc

```
/// <summary>
/// Calculer la durée total du stationnement puis le montant total payé
/// </summary>
1 référence
public void Calc()
{
    isParked = false;
    ExitTime = DateTime.Now;
    TimeSpan duration = GetCurrentParkedTime();
    totalPrice = (int)duration.TotalSeconds;
}
```

Cette méthode est exécutée quand le véhicule sort du parking. Le tarif horaire est calculé en fonction du nombre de seconde que le véhicule est resté dans le parking (un parking très couteux).

GetCurrentParkedTime

Cette méthode qui fait 2 lignes de code permet simplement de calculer le temp de stationnement actuel (utiliser pour les statistiques quotidiennes).

Tests

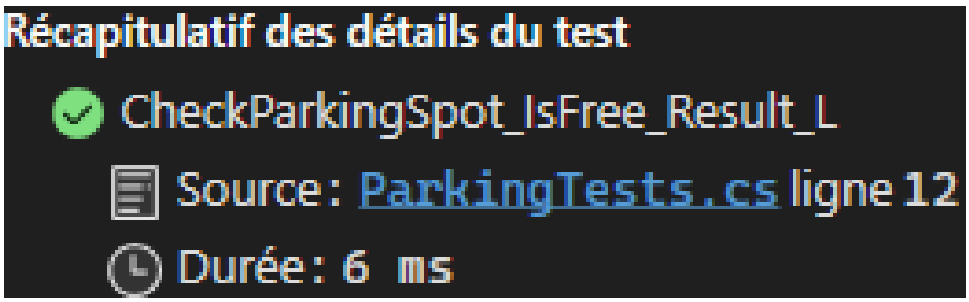
CheckParkingSpot_IsFree_Result_L

La valeur entrée dans ce test était = 0. Puis dans « CheckParkingSpot »

0 = « L » (libre)

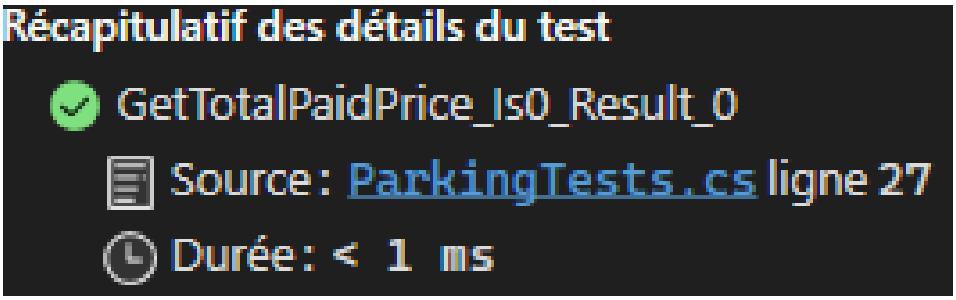
1= « X » (occupé)

Default = « E » (erreur)



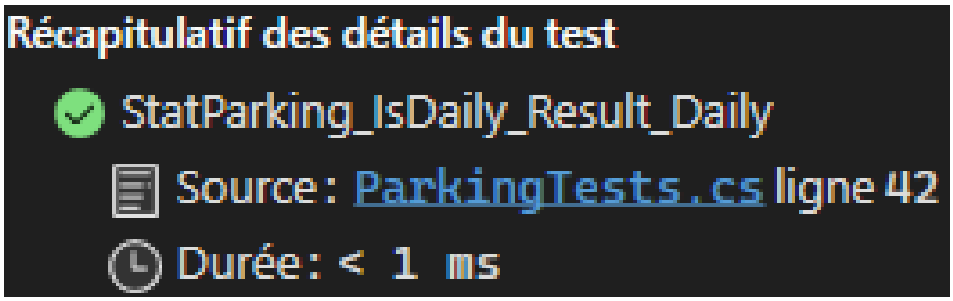
GetTotalPaidPrice_Is0_Result_0

La liste des voitures qui ont quitté le parking était vide, ce qui veut dire qu'aucun montant n'a été payé.



StatParking_IsDaily_Result_Daily

Les statistiques afficher étaient les statistiques du jour (le résultat attendu). L'autre possibilité de statistiques à être afficher et l'historique de transaction.



Conclusion

Bilan personnel

Sur ce projet je trouve que j'ai fait un bon travail à comprendre comment utiliser les objets, et comment les manipulés. Mais je trouve que j'aurais pu faire un meilleur travaille à rédiger mon rapport.

À part ça je suis satisfait, j'ai évité le plus possible la redondance dans mon code, j'ai bien commenté, bien nommé mes variables, etc...

Utilisiation de l'IA

J'ai utilisé l'IA pour m'aider à rédiger mon rapport (4 octobre 2026).

Le premier prompt (ChatGPT) utilisé était celui-ci : "Je dois faire une documentation sur word pour un projet en C# console (qui utilise des classe). Puis je voulais savoir comment je suis censé rédiger cette documentation sous le chapitre « réalisation » ?"

L'IA m'a répondu avec des exemples de comment je pourrais documenter, et comment ne pas le faire. Ducoup après avoir lus ça réponse je lui ait posé une autre question sous ce prompt : "Ducoup si j'ai par exemple 3 méthodes (qui son lier). Je peux très bien expliquer seulement 2 si la troisième méthode par exemple efface simplement ce que l'utilisateur a écrit ?"

Elle m'a répondu que c'était une bonne façon de documenter un chapitre "réalisation". Le chat c'est terminer à cette dernière réponse.

À part l'utilisation de l'IA pour ça, je ne l'ai pas utilisé du tout dans tout mon projet. De la première ligne de code jusqu'au dernier commentaire a été fait entièrement par moi.

Bilan des fonctionnalités

Listes des fonctionnalités présentes et leur état de fonctionnement :

Fonctionnalité	État de fonctionnement
Ajouter un véhicule	100% fonctionnel
Rechercher un véhicule	100% fonctionnel
Retirer un véhicule	100% fonctionnel
Afficher l'état du parking	100% fonctionnel
Afficher les stats du jour	100% fonctionnel
Afficher l'historique des transactions	100% fonctionnel